



420-211-H26

Applications Web

Épreuve finale

Projet intégrateur

Prénom & Nom :

Matricule :

Signature :

Contexte et objectif

Cette épreuve finale vise à évaluer votre capacité à mobiliser les notions abordées durant la session dans le cadre du développement d'une application frontend à l'aide de **React**. Vous devrez démontrer votre compréhension des concepts fondamentaux, notamment la **structuration en composants**, la **gestion de l'état**, la **navigation** entre pages, ainsi que l'**intégration de données** provenant d'une API externe.

Le projet est réalisé de manière **individuelle** sur une **période de trois semaines**.

Description générale du projet

Dans le cadre de ce projet, vous devrez concevoir et développer une application web permettant de **consulter, rechercher et organiser une collection d'items provenant d'une API publique**. Les items peuvent appartenir à différents domaines (par exemple : personnages fictifs, joueurs sportifs, créatures, etc.), selon l'API choisie.

L'application devra offrir une expérience de navigation simple et cohérente, en permettant à l'utilisateur de parcourir une liste d'items, d'effectuer des recherches, de consulter les détails d'un item spécifique et de gérer une liste de favoris.

Structure attendue de l'application

L'application devra être structurée autour d'au moins trois pages principales :

Page 1 - Accueil (*home page*)

La première page correspond à la page d'accueil. Elle doit contenir une section de type « Hero » présentant le nom de l'application ainsi qu'un élément visuel pertinent. Une courte description de l'application doit également être incluse afin d'expliquer son objectif à l'utilisateur.

Page 2 - Liste d'items (recherche)

La deuxième page est dédiée à l'affichage des items. Elle doit présenter une **liste d'éléments sous forme de cartes visuelles** (*card list*). Un champ de saisie (*input*) doit permettre à l'utilisateur d'effectuer une recherche afin de filtrer les résultats. Chaque carte doit afficher une image ainsi qu'une information de base sur l'item et permettre, par interaction, d'accéder à une vue détaillée.

Page 3 - Favoris

La troisième page regroupe les **items favoris**. Elle doit afficher uniquement les éléments que l'utilisateur a explicitement ajoutés à ses favoris. Ces éléments doivent être présentés de manière similaire à la page de liste, afin d'assurer une cohérence visuelle et fonctionnelle.

Page de détails d'un item (ou modale)

Une vue de détail doit également être accessible pour chaque item, soit sous la forme d'une page distincte, soit sous la forme d'une modale. Cette vue doit présenter **au minimum cinq propriétés** de l'item, incluant **obligatoirement une image**.

Exigences fonctionnelles

Certaines fonctionnalités sont obligatoires et doivent être respectées afin que le projet soit considéré comme conforme.

1. Composants globaux

Toutes les pages de l'application doivent inclure :

- Une **barre de navigation** permettant d'accéder aux différentes pages de l'application (accueil, liste, favoris) ;
- Un **pied de page (footer)** affichant le nom de l'application, le nom de l'API utilisée ainsi que votre nom.

2. Gestion des favoris

La gestion des favoris constitue une fonctionnalité centrale du projet. Chaque carte affichée dans les pages de liste et de favoris doit inclure un bouton permettant d'ajouter ou de retirer un item de la liste des favoris. Le comportement attendu est le suivant :

- Lorsqu'un item est ajouté aux favoris, il apparaît dans la page dédiée ;
- Lorsqu'un item est retiré, il disparaît immédiatement de cette page.

*** Un bonus sera accordé si les données relatives aux favoris sont sauvegardées/persistées à l'aide du **localStorage** (de manière à être conservées après rechargement de l'application).*

3. Utilisation d'une API

Vous devez utiliser une API publique permettant de récupérer dynamiquement les données affichées dans votre application.

APIs suggérées

- [PokéAPI](#) – API de pokémons
- [Jikan API](#) – API d'animes
- [Dragon Ball](#) – API des personnages de l'univers Dragon Ball
- [TheAudioDB](#) – API de musique (recherche par artiste ou albums)
- [OMDb API](#) – API de films (nécessite une inscription et l'obtention d'une clé d'API)

Il est également possible d'utiliser une autre API, **sous réserve d'approbation préalable de l'enseignant**. L'API choisie doit impérativement offrir les fonctionnalités suivantes :

- Récupération d'une liste d'items ;
- Accès aux détails d'un item via un identifiant unique ;
- Possibilité d'effectuer une recherche ;
- Présence d'**au moins cinq propriétés par item, incluant une image**.

Afin de vous aider à trouver d'autres APIs, vous pouvez consulter le répertoire suivant : [Free Public APIs](#). Ce site propose une large collection d'APIs gratuites classées par catégories.

Important

Il est de votre responsabilité de lire attentivement la documentation de l'API choisie afin de comprendre son fonctionnement et les modalités d'utilisation. Certaines APIs nécessitent une inscription et l'obtention d'une clé d'accès (API key), tandis que d'autres imposent des limites sur le nombre de requêtes (quotas).

À ce sujet, il est recommandé de porter une attention particulière à votre utilisation des appels API durant le développement. Par exemple, l'activation de la sauvegarde automatique dans votre environnement de

développement (notamment dans VSCode) peut entraîner des rafraîchissements fréquents de l'application et ainsi consommer inutilement votre quota de requêtes. Il est donc conseillé de désactiver cette fonctionnalité au besoin.

Modalités de remise

Le projet doit être **remis sous forme de dépôt Git**.

- Le travail doit être réalisé dans votre dépôt personnel sur GitHub.
- Une branche nommée "projet" doit être utilisée spécifiquement pour cette épreuve.
- Des commits réguliers sont demandés et constituent une partie des points attribués. Une pénalité est attribuée aux travaux remis sur un seul commit.
- Un fichier **README.md** doit obligatoirement être inclus à la racine du projet. Ce document doit contenir une description claire de l'application développée ainsi qu'une planification du travail répartie sur trois semaines, détaillant les principales tâches prévues pour chaque période.

Un exemple de *README* est disponible à l'annexe de l'énoncé. Cet exemple est fourni à titre indicatif: le contenu et la structure peuvent varier, mais toutes les sections demandées doivent être présentes et suffisamment détaillées.

Suivi et démonstrations

Une démonstration individuelle aura lieu à la fin de la session (semaine 15ES/ESO). Celle-ci vise à évaluer votre compréhension de l'application développée ainsi que votre capacité à expliquer vos choix d'implémentation.

La démonstration aura une **durée d'environ 10 minutes** et devra présenter les principales fonctionnalités de votre application ainsi que certains éléments de votre code.

Les modalités sont les suivantes :

- Les 9 premiers étudiants inscrits effectueront leur démonstration lors de la dernière séance de cours (semaine 15ES).
- Les autres étudiants devront prendre rendez-vous avec l'enseignant pour effectuer leur démonstration en dehors des heures de cours.
- **La date limite pour la prise de rendez-vous est fixée au 20 mai.**
- **Toutes les démonstrations devront être complétées au plus tard le 22 mai.**

Il est de votre responsabilité de vous inscrire ou de planifier votre rendez-vous dans les délais prescrits. **Aucune démonstration ne sera acceptée après la date limite.**

Évaluation

L'évaluation du projet repose à la fois sur la **qualité du produit final** et sur votre **capacité à en expliquer le fonctionnement**.

La note attribuée au projet pourra être ajustée par un facteur multiplicatif en fonction de votre performance lors de la démonstration :

- Une compréhension complète et des explications claires maintiendront la note initiale ;
- Les lacunes partielles entraîneront une réduction modérée ;
- Une compréhension insuffisante pourra entraîner une réduction plus importante, pouvant aller jusqu'à 60 % de la note obtenue.

Bonifications

Des points bonus peuvent être accordés pour l'ajout de fonctionnalités supplémentaires pertinentes. À titre d'exemples :

- Intégration d'un carrousel ;
- Ajout de filtres avancés pour la recherche ;
- Utilisation d'une bibliothèque de styles (par exemple Tailwind ou Bootstrap) ;
- Implémentation d'un thème sombre ;
- Déploiement de l'application en ligne ;
- Utilisation du `localStorage` pour faire persister les favoris ;
- Implémentation d'un système de pagination ;
- Conception d'une API personnalisée.

Exemple fourni

Un exemple d'application complète est mis à votre disposition [au lien suivant](https://disneyapi.dev/) afin d'illustrer les attentes fonctionnelles et le niveau de complexité visé. Cet exemple utilise l'API de Disney (<https://disneyapi.dev/>) pour récupérer les informations des personnages du monde de Disney. **Cet exemple doit être utilisé comme référence uniquement et ne doit en aucun cas être reproduit.**

Consignes générales

Le projet doit être réalisé **individuellement**. Vous êtes **responsable de l'ensemble du code soumis**. Toutefois, il est permis de s'inspirer de ressources externes (documentation, tutoriels, forums, exemples de code, etc.), comme cela se fait couramment en contexte professionnel, mais cette pratique est encadrée par les exigences suivantes :

- Toute portion de code inspirée ou adaptée d'une source externe **doit être clairement identifiée dans votre code à l'aide d'un commentaire incluant l'URL de la source** ;
- Vous devez **démontrer une compréhension réelle du code utilisé** et être en mesure de **l'expliquer de façon claire lors de la démonstration**.

L'utilisation de code externe sans attribution ou sans compréhension adéquate pourra être considérée comme une forme de plagiat. Toute situation de plagiat ou de collaboration non autorisée sera déclarée à l'administration et traitée conformément aux règles en vigueur.

Grille de correction

Critère	Points
1. Structure générale et architecture <i>Structure du projet claire et logique (dossiers, composants)</i> <i>Découpage pertinent en composants réutilisables</i> <i>Utilisation adéquate des hooks (useState, useEffect, etc.)</i>	15
2. Navigation et routing <i>Implémentation fonctionnelle du routing (3 pages principales)</i> <i>Navigation fluide via la barre de navigation</i> <i>Gestion correcte des routes (ex : détails via ID)</i>	10
3. Page d'accueil <i>Présence d'un Hero (titre + visuel)</i> <i>Description claire de l'application</i>	5
4. Page de liste et recherche <i>Affichage des items sous forme de cards</i> <i>Intégration correcte des données depuis l'API</i> <i>Fonctionnalité de recherche fonctionnelle</i>	15
5. Page de détails <i>Affichage d'au moins 5 propriétés pertinentes</i> <i>Présence d'une image</i> <i>Accès correct depuis les cards</i>	10
6. Gestion des favoris <i>Bouton ajouter/enlever fonctionnel sur toutes les cards</i> <i>Ajout correct des items aux favoris</i> <i>Suppression correcte des items</i> <i>Synchronisation entre pages (liste ↔ favoris)</i>	20
7. Composants communs <i>Barre de navigation présente sur toutes les pages</i> <i>Footer conforme (nom app + API + étudiant)</i>	5
8. Intégration de l'API <i>Appels API fonctionnels et bien intégrés</i> <i>Gestion minimale des erreurs ou états (loading, etc.)</i>	10
9. Qualité du code <i>Code lisible et bien formaté</i> <i>Nommage clair des variables et fonctions</i> <i>Commentaires pertinents (incluant sources externes)</i>	5
10. README.md <i>Description claire du projet</i> <i>Planification sur 3 semaines</i>	5
Bonus (optionnel – jusqu'à + 10 points) <i>Implémentation d'une (ou plusieurs) fonctionnalité.s supplémentaire.s pertinente.s</i>	
TOTAL	100

Ajustement – Démonstrations (facteur multiplicatif)

La note finale sera ajustée selon la qualité des démonstrations :

- Excellente compréhension $\rightarrow \times 1.0$
- Bonne compréhension $\rightarrow \times 0.9 - 0.99$
- Compréhension partielle $\rightarrow \times 0.75 - 0.89$
- Compréhension faible $\rightarrow \times 0.60 - 0.74$

Annexe – exemple de README.md attendu

Nom de l'application

Description

Cette application web a été développée dans le cadre du projet final du cours 420-211 - Application Web.

L'objectif de l'application est de permettre aux utilisateurs de rechercher et consulter des Pokémon à partir de l'API PokéAPI. L'utilisateur peut parcourir une liste de Pokémon, effectuer une recherche par nom, consulter les détails de chaque Pokémon et sauvegarder ses favoris.

Fonctionnalités principales

- Affichage d'une liste de Pokémon sous forme de cartes
- Recherche de Pokémon par nom
- Consultation des détails d'un Pokémon (image, types, statistiques, etc.)
- Ajout et suppression de favoris

API utilisée

- Nom : PokéAPI
- URL : <https://pokeapi.co/>

Planification du projet

Semaine 1

- Initialisation du dépôt GitHub (branche “projet”)
- Mise en place de la structure des composants
- Configuration du routing (pages principales)
- Choix d'une API publique et rédaction du README.md
- Implémentation de données “en dur” (mock) basé sur le format de l'API

Semaine 2

- Implémentation de la page de liste (cards)
- Ajout de la fonctionnalité de recherche
- Création de la page de détails
- Intégration des données de l'API

Semaine 3

- Implémentation des favoris (ajout / suppression)
- Création de la page des favoris et de la logique d'ajout et de suppression
- Améliorations visuelles (UI / CSS / gestion de l'état de chargement et des erreurs)
- Tests et correction de bugs

Auteur : [Votre nom]